

Wind-Aware Service Provisioning Strategy for Multi-Package Drone Delivery

Jia Xu , *Member, IEEE*, Hao Liu, Xiao Liu , *Senior Member, IEEE*, Azadeh Ghari Neiat ,
Xuejun Li , *Member, IEEE*, and Yun Yang , *Senior Member, IEEE*

Abstract—In recent years, drone delivery has drawn significant attention for its promising potential in solving the last-mile delivery problem. As a novel service paradigm, Drone-as-a-Service (DaaS) is emerging as an effective way to address service provisioning problems within complex delivery networks. However, existing DaaS composition frameworks often fail to consider the sharing of delivery services and are not applicable to multi-package drone delivery tasks. Meanwhile, as one of the most critical real-world environmental factors for drones, the impact of dynamic wind conditions is not adequately considered by existing studies. This may lead to Quality of Service (QoS) degradation of delivery services. To address the above issues, in this paper, we propose a wind-aware service provisioning strategy for multi-package drone delivery. Given the advantages of Edge Computing (EC) in handling such dynamic factors due to its low latency and high reliability, we first establish a spatio-temporal DaaS model based on service sharing according to the edge-based drone delivery system. Then, we propose a novel wind-aware drone delivery service provisioning strategy for multi-package delivery to minimize energy consumption of drones. The proposed strategy consists of two phases: service sharing and service composition. In the service sharing phase, the service sharing plan is generated by an improved genetic algorithm. In the service composition phase, the edge server dynamically generates the service composition plan through our proposed policy iteration based DaaS composition method. Experimental results using real delivery network and wind data demonstrate that our strategy is able to reduce delivery energy consumption of drone by about 10.1%.

Index Terms—Intelligent logistics, drone delivery, drone-as-a-service, service provisioning, edge computing.

I. INTRODUCTION

LAST-MILE delivery represents a pivotal component of intelligent logistics systems. It is closely related to the cost of service providers and the experience of end users. Service providers want to deliver packages as quickly as possible at lower cost, while end users prefer to use services with lower cost and shorter time. This demand for delivery service has driven the exploration of innovative last-mile delivery solutions, with drone technology emerging as a promising candidate [1]. For example, Amazon Prime Air project can deliver medication to homes by drone in 60 minutes [2]. The Wing Delivery Network logistics platform developed by Alphabet Wing supports large-scale drone delivery tasks [3]. Compared with traditional delivery methods, drone delivery has the advantages of low cost, fast delivery, high flexibility, and low carbon emissions. However, due to the complexity of the delivery environment and the limitations of the drone itself, the development of an efficient delivery solution is still a challenging question.

The service paradigm provides a flexible and scalable approach to modeling complex constraints in delivery networks, enabling efficient and reliable solutions. It abstracts both functional and non-functional attributes of drones as Drone-as-a-Service (DaaS) [4]. Delivering packages along a line segment in delivery networks represents a drone delivery service operating under constraints. Given a delivery network, DaaS composition is the process of selecting the optimal drone services to reduce delivery time and improve other QoS (Quality of Service) dimensions such as energy consumption. These services form an aerial path from a given source to the destination. Typically, a single delivery service request cannot be fulfilled by just one delivery service. Therefore, the significant challenge lies in generating a delivery service provisioning plan that minimizes delivery time and energy consumption, based on the characteristics of the delivery service request and the current state of the resources.

While most existing studies focus on single-package drone delivery, multi-package drone delivery refers to the delivery of multiple packages by a drone in a single task. This delivery approach can fully utilize the payload capacity of drones and use fewer drones to complete the delivery tasks [5]. Compared with single-package delivery, multi-package delivery incurs lower energy consumption in many cases. For example, assigning several packages with the same or close destinations to a single

Received 11 November 2024; revised 4 June 2025; accepted 18 July 2025. Date of publication 24 July 2025; date of current version 9 October 2025. This work was supported in part by Young Scientists Fund of the National Natural Science Foundation of China under Grant 62302005, in part by China Postdoctoral Science Foundation under Grant 2023M740014, and in part by Anhui Provincial Natural Science Foundation, China under Grant 2308085QF219. An earlier version of this paper was presented in part at the Work-in-Progress (WIP) of IEEE International Conference on Web Services in 2024: Wind-Aware Service Provision Strategy for Multi-Package Drone Delivery [DOI: 10.1109/ICWS62655.2024.00169]. (Corresponding authors: Xiao Liu; Xuejun Li.)

Jia Xu is with the School of Internet, Anhui University, Hefei 230601, China, and also with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210093, China (e-mail: xujia@ahu.edu.cn).

Hao Liu and Xuejun Li are with the School of Computer Science and Technology, Anhui University, Hefei 230601, China (e-mail: hl@stu.ahu.edu.cn; xjli@ahu.edu.cn).

Xiao Liu and Azadeh Ghari Neiat are with the School of Information Technology, Deakin University, Geelong, VIC 3220, Australia (e-mail: xiao.liu@deakin.edu.au; azadeh.gharineiat@deakin.edu.au).

Yun Yang is with the Department of Computing Technologies, Swinburne University of Technology, Melbourne, VIC 3122, Australia, and also with the School of Computer Science and Technology, Anhui University, Hefei 230601, China (e-mail: yyang@swin.edu.au).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TSC.2025.3592374>, provided by the authors.

Digital Object Identifier 10.1109/TSC.2025.3592374

drone for delivery may reduce the overall delivery distance and hence the total energy consumption. As a form of service sharing, this approach allows a service instance to serve multiple user requests at the same time.

However, current DaaS composition frameworks are not applicable to multi-package drone delivery as they often overlook the sharing of drone services among user requests. Furthermore, existing approaches do not adequately consider the impact of dynamic wind conditions (as one of the most critical real-world environmental factors), particularly when selecting delivery stations for recharging or waiting. This may increase energy consumption of drones. With limited battery capacity, the drones may need to recharge to reach their destinations during the delivery process. Also, they may need to wait at a delivery station when encountering strong winds. The decisions regarding recharging and waiting refer to when to recharge or wait and which delivery station to choose. For the same delivery path, choosing different recharging or waiting stations may result in different energy consumption of drones. This is because drones may encounter different wind conditions. Therefore, path selection, recharging decision, and waiting decision should ensure drones avoid flying against the wind as much as possible. Moreover, the dynamic changes in wind conditions can greatly influence the effectiveness of service provisioning strategies. In contrast to these limitations, our work proposes a wind-aware DaaS service provisioning strategy for multi-package delivery. This strategy enables drones to dynamically adapt to varying wind conditions and make intelligent decisions about when and where to recharge or wait, thereby reducing drone energy consumption during delivery.

Meanwhile, to facilitate the services for drone delivery systems in the complex environment, many real-time computational tasks are required. However, it is very difficult for centralized cloud computing to support real-time computational tasks. In contrast, edge computing provides more responsive computing services at the edge of the network, enabling real-time planning under dynamic factors [6]. Edge computing can effectively address the challenges caused by dynamic factors in the delivery process, so as to facilitate the implementation of service provisioning strategies in complex delivery environments. Therefore, edge computing is now widely accepted as a promising computing paradigm for DaaS [7].

There is a body of related work that investigates the drone delivery problem from two perspectives: drone routing and scheduling, and delivery service composition. Some research work models the drone delivery problem as the Travelling Salesman Problem (TSP) or the Vehicle Routing Problem (VRP) [8]. This type of work is applicable to multi-package drone delivery tasks, but does not involve complex delivery networks. It focuses on fixed routing or deterministic optimization and lacks integration with dynamic wind conditions. Some studies focus on service-based approaches for drone or swarm-based drone delivery [9]. The service-based approach models the drone path planning and scheduling problem as a service composition and service selection problem. Orchestrated through a service-oriented framework, service composition and selection can more effectively deal with the constraints and dynamics of the delivery

environment. Existing approaches account for dynamic factors in the delivery networks but normally focuses on single-package drone delivery scenarios. Moreover, without considering optimal recharging and waiting decisions, current approaches often lead to an unnecessary increase in energy consumption. This may result in failure to meet QoS requirements. Aiming to address problems in current research, we investigate the delivery service provisioning problem in the edge-based last-mile drone delivery system. Our approach focuses on multi-package drone delivery in complex delivery networks, as a form of service sharing problem. Meanwhile, we consider the impact of dynamic wind conditions on path selection, recharging, and waiting to reduce the delivery energy consumption. To the best of our knowledge, our work is the first to jointly optimize multi-package delivery under dynamic environmental constraints. The main contributions of this paper are summarized as follows:

- A spatio-temporal DaaS model based on service sharing is established according to an edge-based drone last-mile delivery system.
- A wind-aware DaaS service provisioning strategy for multi-package delivery (named MW-DSP) is proposed to minimize the energy consumption of drones. The MW-DSP strategy consists of two phases: service sharing and service composition.
- An improved genetic algorithm (named ADS-GA) is proposed for service sharing. The ADS-GA algorithm includes an agglomerative clustering based population initialization method and a directed mutation mechanism, and incorporates neighborhood search.
- A policy iteration based DaaS composition method (named PIDC) is proposed for dynamically composing near optimal DaaS services.

The rest of the paper is organized as follows: Section II presents related work in the areas of drone delivery, DaaS, service selection and composition. Section III first presents the delivery system model. Then the wind and drone energy consumption models as well as the service model are presented. Finally, the service provisioning problem is defined. Section IV introduces the MW-DSP strategy and details our proposed ADS-GA algorithm and PIDC method. Section V evaluates our strategy through simulation experiments. Section VI concludes this paper.

II. RELATED WORK

In recent years, the drone delivery problem has attracted extensive attention from researchers. In this section, we review related research in drone routing and scheduling, drone-based delivery service, and service selection and composition.

Several studies have modeled the drone delivery problem as TSP or its variants. Tu et al. [10] studied the truck-drone cooperative delivery problem and modeled it as a multi-drone traveling salesman problem. Cavani et al. [11] developed a compact mixed-integer linear programming model for the TSP with multiple drones. They presented an exact decomposition method based on the branch-and-cut algorithm. Lei et al. [12] proposed an improved variable neighborhood search algorithm

for the parallel drone scheduling traveling salesman problem, aiming to reduce the total completion time of delivery tasks.

However, TSP is not suitable for modeling the drone delivery problem due to the limited payload of drones and the impact of dynamic environmental factors. Dorling et al. [13] proposed two multitrip VRPs to model the drone delivery problem. The experimental results demonstrated the importance of reusing drones in drone delivery systems. Chiang et al. [14] developed an uncrewed aerial vehicle (UAV) green routing model and designed a genetic algorithm for solving large problem instances. Yuan et al. [15] investigated a scheduling problem with heterogeneous logistics UAVs in urban environments. They proposed a framework based on genetic algorithm for solving the scheduling problem. Sajid et al. [16] presented a joint optimization framework for solving UAV-routing and UAV-route scheduling problems. The proposed hybrid genetic simulated annealing algorithm and the UAV-oriented scheduling algorithm were used to solve these two problems respectively. Sorbelli et al. [17] focused on drone delivery under variable wind conditions and defined the mission-feasibility problem. They modeled the wind through a time-dependent cost graph. Sorbelli et al. [18] investigated the effect of the wind in a truck-drone tandem delivery system and proposed a minimum-energy drone-trajectory problem. Shi et al. [19] considered the recharging constraints of drones and designed an improved adaptive large neighborhood search algorithm. Mahdi et al. [20] analyzed the impact of serving multiple customers in single drone sortie versus several sorties delivering a single package. The experimental results showed that multi-package payload of drones can show benefits under some conditions. However, the proposed method does not consider dynamic environmental factors such as wind.

Many research modeled the drone delivery problem using a service paradigm that abstracts routing into service composition under complex constraints. Shahzaad et al. [4] focused on the spatio-temporal aspects of drone delivery services and the dependencies between QoS attributes. They proposed a composition framework for drone-based package delivery services. A heuristic-based multi-armed bandit method was proposed to compose drone services in [21]. The authors considered drone recharging constraints and the stochastic arrival of drone services. Alkouz et al. [22] studied the swarm-based drone delivery problem and proposed a swarm-based DaaS (SDaaS) composition framework with in-flight energy recharging. They proposed an enhanced A* (A-star) algorithm for service composition. Shahzaad et al. [23] focused on the recharging pad availability and the variations of wind to design a robust DaaS composition framework to minimize the delivery time. Shahzaad et al. [24] considered dynamic weather conditions and proposed a deterministic lookahead algorithm to build an initial offline composition plan. To adapt to the changes in service behavior, they developed a heuristic-based resilient service composition algorithm. Chu et al. [25] considered drone relay delivery and proposed a mobile edge computing based holistic service provisioning strategy for drone delivery. Hamdi et al. [26] focused on the uncertainty of weather and proposed a DaaS service model based on the dynamic spatio-temporal characteristics of drones. Shahzaad et al. [27] investigated the multi-package drone

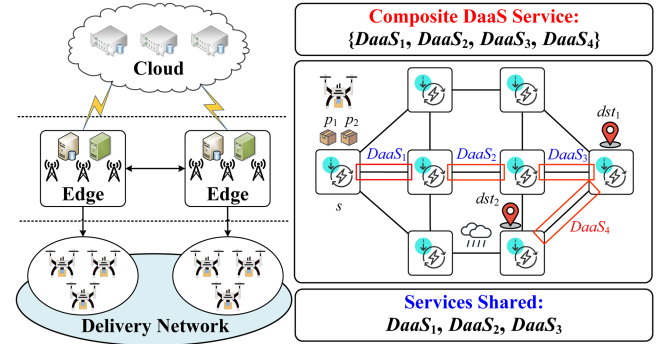


Fig. 1. System model of edge-based drone last-mile delivery system.

delivery problem under recharging constraints and proposed a new service-oriented architecture. However, the proposed method is not concerned with how DaaS services are shared among user requests.

Meanwhile, many studies have investigated different service selection and composition problems. Lima et al. [28] proposed a QoS-aware service selection method that considered service sharing. The method included a novel model for the deployment of multiple choreographies and considered the impact of service sharing on QoS. Zhang et al. [29] constructed a multi-objective constraint optimization model for service level agreement aware service composition problem. They considered the resource sharing property. Ren et al. [30] modeled the constraint satisfaction service composition problem as a Markov Decision Process (MDP). They considered the uncertainty in QoS and service behavior. Zhao et al. [31] proposed an adaptive composition mechanism leveraging Computation Tree Logic (CTL) specifications. They evaluated the dependencies between services with multiple QoS attributes.

Most existing research on drone routing and scheduling focuses on point-to-point delivery between warehouses and customers. Service-based approaches are limited to providing solutions for single-package delivery by drones. Meanwhile, both types of research overlook the impact of recharging and waiting under dynamic wind conditions on drone energy consumption. In this paper, we aim to provide an overall solution for multi-package delivery under dynamic wind conditions. Our focus is on finding the near optimal service sharing and service composition plans to minimize the energy consumption.

III. PROBLEM FORMULATION

In this section, we first describe the system model of the edge-based drone last-mile delivery system. Then, wind model, drone energy consumption model, and the spatio-temporal DaaS model based on service sharing are presented. Finally, the service provisioning problem is defined. Table I lists and explains some important symbols used in this paper.

A. System Model

In this paper, we focus on delivery scenarios with fixed delivery stations and predefined aerial paths. As shown in Fig. 1,

TABLE I
IMPORTANT SYMBOL AND EXPLANATION

Symbol	Definition and description
G	Delivery network
e_{ij}	Edge from vertex i to vertex j
d_{ij}	Length of edge e_{ij}
W	Wind conditions of delivery network
W_P	Predicted wind conditions of delivery network
$w_{ij}^t = (w_s, w_d)$	Represents wind speed and wind direction at edge e_{ij} in time slot t
w_o	Angle between wind direction and drone's flight direction
t_{ij}	Time required for drone to fly from node i to adjacent node j
l_t	Length of a time slot
FP	Required power of drone in stable flight
E_{ij}^t	Energy consumption of the drone flying from node i to node j at time t
$Sim(x, y)$	Similarity function of packages
θ	Weight of probability reduction of the corresponding action in the policy update
φ	Threshold for determining dynamic processing

we abstract a drone delivery scenario that includes cloud servers, edge servers, and fixed delivery stations, based on the real-world Antwork's last-mile delivery network (ADNET) [32]. Assume that a logistics service provider uses several different types of drones to deliver packages in the delivery network. Each delivery station is equipped with drones and recharging equipment. All stations can be used as the source or the destination for packages.

A drone flies along line segments between stations in the delivery process. During this period, it has the option to choose between recharging or waiting at a particular station. Drones can carry multiple packages to multiple destinations. Fig. 1 illustrates an example of multi-package drone delivery. The drone carries package p_1 and package p_2 from station s to destinations dst_1 and dst_2 , at which time the two packages share part of the DaaS services. After composite DaaS service $\{DaaS_1, DaaS_2, DaaS_3\}$ delivers package p_1 to its destination dst_1 , service $DaaS_4$ delivers package p_2 to its destination dst_2 . Appendix A discusses the service provisioning process in detail. Due to the difficulty in predicting the availability of recharging equipment, the waiting time for recharging is ignored in this paper to simplify the problem. The delivery network is assumed to be within the service coverage of the cloud and edge servers. The cloud server provides computing services for each delivery station. The edge server provides computing services for drones performing delivery tasks.

The delivery network is represented as an undirected connected graph G . The vertices in graph represents the delivery stations, and the edges represents the skyway. The weight d_{ij} of the edge e_{ij} represents the length of the skyway between two stations. $P_i = \{p_1, p_2, \dots, p_n\}$ represents the set of packages that station i needs to deliver within the current time range. p_i is a 3-tuple $\langle id, dst, \omega \rangle$, representing the package's unique identifier id , destination dst , and weight of the package ω ,

respectively. $D_{type} = \{D_1, D_2, \dots, D_n\}$ denotes the collection of drone types. D_i is the relevant attributes of a certain type of drone, including the maximum payload capacity L , flight speed v , and battery capacity B , and so on.

B. Wind and Drone Energy Consumption Models

Wind is a major environmental factor affecting drone performance and flight behavior. The headwinds significantly increase the energy consumption of drones. Moreover, wind conditions change dynamically over time. Therefore, the development of service provisioning strategies needs to consider the impact of dynamic wind conditions. In real-world scenarios, wind data is often collected at fixed time intervals. We discretize the wind in the time dimension, assuming that the wind changes at every fixed time slot [17]. The wind conditions across the delivery network in the future t time slots are represented as a sequence $W = \{W^0, W^1, \dots, W^t\}$. W^i represents the wind conditions in each region of the delivery network in the time slot i . $w_{ij}^t = (w_s, w_d) \in W^t$ represents the wind conditions at edge e_{ij} in the time slot t . w_s and w_d are the wind speed and wind direction, respectively. The relative wind direction w_o is defined as the angle between the wind direction and the drone's flight direction.

In practice, it is feasible to achieve minute-level predictions of short-term wind conditions [33]. We assume that the predicted wind conditions W_p across the delivery network in the future t time slots are known. The wind prediction is performed using historical data from sources like OpenWeatherMap [34]. All the drone delivery tasks can be completed within this timeframe. A drone may require several time slots to fly from the current node to an adjacent node. While this assumption holds for the current study, our future work will explore more robust methods. Due to the uncertainty of wind, the predicted wind conditions W_p may differ from the actual wind conditions W during the delivery process. Therefore, our strategy uses the predicted wind conditions for planning before the drone take-off, i.e., in the static phase. During the dynamic phase, the edge server detects the difference between W_p and W . Then, it executes real-time adjustments according to the impact caused by the changes in wind conditions.

According to the energy consumption model, the drone needs to overcome gravity and air resistance during flight [35]. The total thrust required is calculated as follows:

$$F = Mg + F_D \quad (1)$$

where M represents the total weight of the drone and carried packages, g is the coefficient of gravity. F_D is the total drag force of flight. It can be calculated by the following equation:

$$F_D = \sum_i \frac{1}{2} \rho v_a^2 C D_i C A_i \quad (2)$$

where ρ is the air density, v_a denotes the relative airspeed of the drone. $C D_i$ and $C A_i$ represent the drag coefficient and cross-sectional area of the corresponding component, respectively. Denoting that the drone speed is v , the wind speed is w_s , and the relative wind direction is w_o . The drone's relative

airspeed v_α can be calculated by the following equation:

$$v_\alpha = \sqrt{(v - w_s \cos w_o)^2 + (w_s \sin w_o)^2} \quad (3)$$

The required power of the drone in stable flight is:

$$FP = \frac{F(v \sin \alpha + v_i)}{\eta} \quad (4)$$

where η is the overall power efficiency of the drone. α is the pitch angle of the drone during stable flight and v_i represents the induced velocity required. α and v_i are given as follows:

$$\alpha = \tan^{-1} \left(\frac{F_D}{Mg} \right) \quad (5)$$

$$v_i = \frac{2T}{\pi n \beta^2 \rho \sqrt{(v \cos \alpha)^2 + (v \sin \alpha + v_i)^2}} \quad (6)$$

where n is the number of rotors, β is diameter of one rotor. The time required for the drone to fly from node i to adjacent node j is:

$$t_{ij} = \frac{d_{ij}}{v} \quad (7)$$

Since the wind conditions may be different in each time slot, the drone may have different power in each time slot. Assume that $\{FP_t, FP_{t+1}, \dots, FP_n\}$ represents the power per time slot when the drone flies from node i to adjacent node j . $n = \lceil \frac{t_{ij}}{l_t} \rceil + t - 1$ denotes the last time slot of the flight, l_t is the length of a time slot. The energy consumption of the drone flying from node i to node j at time t can be computed by the following equation:

$$E_{ij}^t = \sum_{k=t}^{n-1} FP_k l_t + FP_n \frac{d_{ij} - l_t(n-t)v}{v} \quad (8)$$

C. Spatio-Temporal DaaS Model Based on Service Sharing

Drone services have strong spatio-temporal attributes. The quality parameters such as delivery energy consumption and availability of services vary with time and space. Therefore, we establish a spatio-temporal DaaS model based on service sharing [4]. The relevant definitions are as follows.

Definition 1. Package Delivery Request (PDR): A PDR is defined as a service request to deliver a package from its source to its destination. It is represented as a 5-tuple $\langle id, rt, src, dst, \omega \rangle$. id is the unique identifier, rt is the request time-stamp. src and dst denote the source and the destination of the package, respectively. ω denotes the weight of the package.

Definition 2. Sharing-based Drone-as-a-Service (DaaS): A DaaS represents a drone delivery service operating under a set of constraints that can deliver a set of packages from a station to an adjacent station. A single DaaS service instance can serve multiple requests at the same time. A DaaS is represented as a 4-tuple $\langle DaaS_{id}, D, DaaS_f, DaaS_q \rangle$. $DaaS_{id}$ is the unique identifier of the drone service. D denotes the drone type of the service. $DaaS_f$ represents the delivery function of the DaaS, where the spatio-temporal attributes of the function are represented as $\langle loc_s, loc_e \rangle$ and $\langle t_s, t_e \rangle$. loc_s and loc_e denote the source and the destination of the DaaS, respectively. t_s and t_e denote the start time and the end time, respectively. $DaaS_q$ is the

relevant quality parameter of the DaaS. The energy consumption of a DaaS service is related to the spatio-temporal attributes and can be represented as:

$$E_{DaaS} = E_{loc_s loc_e}^{t_s} \quad (9)$$

Service sharing enhances resource utilization as well as increases the number of candidate service instances. Nevertheless, unsupervised service sharing may also lead to a decrease in the QoS of the DaaS services. In order to utilize the benefits of service sharing, the service sharing method must select appropriate DaaS services.

Definition 3. Service Sharing Plan (SSP): An SSP is represented as a 3-tuple $\langle SSP_{id}, SSP_p, SSP_q \rangle$. SSP_{id} is the unique identifier of the SSP. $SSP_p = \{\langle PS_1, D_1 \rangle, \langle PS_2, D_2 \rangle, \dots, \langle PS_n, D_n \rangle\}$ represents how DaaS services are shared among PDRs in current time range. It determines which packages will share a set of DaaS services and their drone types. PS_i is a sequence of service requests. D_i represents the type of drone used. $\langle PS_i, D_i \rangle$ represents a sub-route where packages belonging to the PS_i will be delivered sequentially by drone D_i in a single task. The total weight of the packages in the PS_i cannot exceed the maximum payload capacity of the drone D_i . SSP_q is the relevant quality parameter of the SSP. For the same service sharing plan, choosing different delivery paths will lead to different energy consumption. We define the energy consumption of delivering packages along the shortest path as the expected energy consumption of the SSP. Assume $R = \{R^1, R^2, \dots, R^n\}$ contains the path selection information when delivering packages for each sub-route. $R^i = \{path_1, path_2, \dots, path_n\}$ is the set of the path selection for each package within sub-route i . $path_j$ is denoted as a collection of 3-tuple $\langle t_s, loc_s, loc_e \rangle$. It represents the shortest path when delivering the package j and gives the start time of each line segment. The expected energy consumption can be calculated by the following equation:

$$E_{SSP} = \sum_{i=1}^n \sum_{j=1}^{len(PS_i)} \sum_{\langle t_s, loc_s, loc_e \rangle \in R^i(j)} E_{loc_s loc_e}^{t_s} \quad (10)$$

Definition 4. Composite DaaS Service (CS): A CS includes all DaaS services utilized by packages in a sub-route and is denoted as a 3-tuple $\langle CS_{id}, CS_s, CS_q \rangle$. When n service requests in a sub-route share a set of drone delivery services, CS_s is denoted as $\{CDS_1, CDS_2, \dots, CDS_n\}$. CDS_i represents DaaS services selected to provide service for request i after completing request $i - 1$. If a single DaaS service is unable to fulfill the request, multiple DaaS services need to be composed. These DaaS services have temporal and spatial dependencies. CS_q represents the relevant quality parameters of the CS, which are obtained by aggregating the quality parameters of the atomic services. The energy consumption of the CS is given as:

$$E_{CS} = \sum_{i=1}^n \sum_{DaaS \in CDS_i} E_{DaaS} \quad (11)$$

Definition 5. Service Composition Plan (SCP): An SCP includes DaaS services utilized by all packages. It depends on

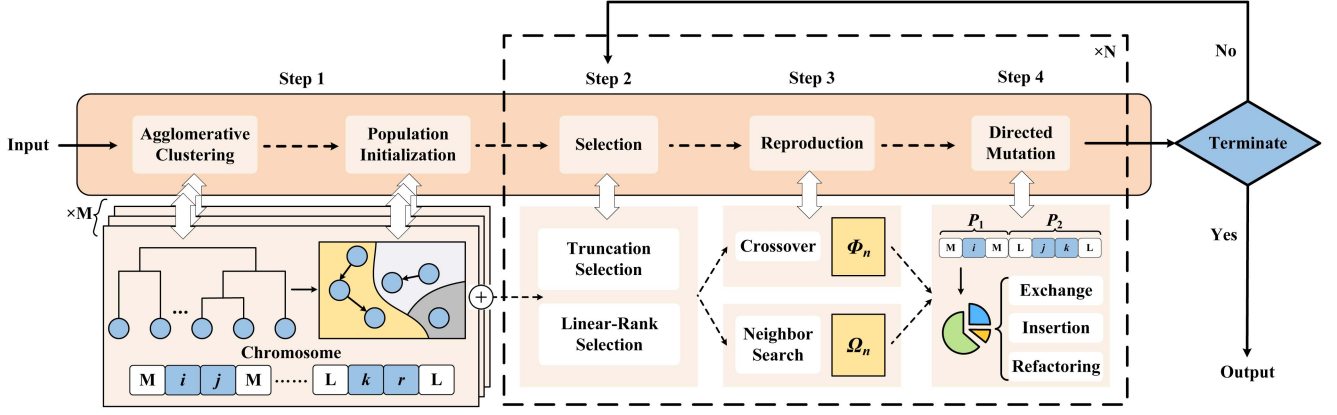


Fig. 2. Overall process of ADS-GA.

an SSP, denoted as a 4-tuple $\langle SCP_{id}, SSP_{id}, SCP_s, SCP_q \rangle$. SCP_{id} is the id of the SCP, SSP_{id} is the id of the SSP on which SCP depends. $SCP_s = \{CS_1, CS_2, \dots, CS_n\}$ denotes the collection of the CS corresponding to each sub-route in the SSP. SCP_q represents the quality parameters, which are obtained by aggregating the quality parameters of all the CS. The energy consumption of the SCP can be represented as:

$$E_{SCP} = \sum_{i=1}^n E_{CS_i} \quad (12)$$

D. Problem Definition

The service provisioning problem is to use several types of drones to complete the delivery tasks and minimize the delivery energy consumption. According to the service model outlined in the preceding subsections, the optimization problem is formulated as follows:

$$\min E = \sum_{i=1}^n \sum_{j=1}^{\text{len}(PS_i)} \sum_{DaaS \in CS_i(j)} E_{locsloc_e}^{ts} \quad (13)$$

The decision variables of the optimization problem are SSP and SCP. The problem is difficult to be solved exactly because the actual wind conditions are unknown. Therefore, we solve the service sharing problem and the service composition problem sequentially to obtain a solution for the service provisioning problem. The service sharing problem refers to determining the SSP that minimizes the expected energy consumption for delivery, i.e., minimizing the E_{SSP} (10). The service composition problem refers to determining the SCP based on the SSP to deliver all the packages. The optimization objective is to minimize the energy consumption of the drones, i.e., minimizing the E_{SCP} (12).

IV. SERVICE PROVISIONING STRATEGY MW-DSP

This section first describes the MW-DSP strategy and then details the proposed ADS-GA algorithm and PIDC method.

MW-DSP strategy consists of two phases: service sharing and service composition. In the service sharing phase, the cloud server obtains the package information and then generates the

SSP through ADS-GA. The packages from the same station within the current time range will be divided into several sub-routes. These packages are then delivered to their destinations by drones via a multi-package delivery plan. In the service composition phase, the edge server composes DaaS for the packages carried by the drone sequentially according to the SSP. The PIDC method is used to compose near optimal DaaS services. Whenever the drone arrives at a new delivery station, the edge server will determine either the path selection, the recharging decision, or the waiting decision.

A. ADS-GA Algorithm

The overall process of ADS-GA is illustrated in Fig. 2, encompassing four steps: population initialization, selection, reproduction, and mutation. First, the algorithm generates an initial population representing a set of SSPs based on agglomerative clustering. Then, the population evolves within the search space to discover better approximate solutions until the stopping criterion is met. To balance the quality and the diversity of the population, ADS-GA employs truncation selection and linear-rank selection. In the selection process, the algorithm retains individuals with superior fitness values and probabilistically eliminates the others. In the reproduction, ADS-GA incorporates neighborhood search to generate new individuals by searching the neighborhoods of good individuals. Where 50% of offspring are generated through crossover operation, while the remaining 50% are generated through neighborhood search. In the mutation, the algorithm performs mutation operation based on a directed mutation mechanism. The poorer gene fragments have a higher probability of mutation. Three types of mutation operators are utilized: exchange, insertion, and refactoring. The following subsections describe the encoding, population initialization method, and genetic operators in detail.

1) *Encoding and Population Initialization*: As shown in Fig. 2, a chromosome is a structured arrangement of the drone type identifier D_i and the package's id . Its fitness value is the expected energy consumption of the corresponding SSP. The drone type identifiers divide the chromosome into several gene fragments, each representing a sub-route. The total weight of

the packages in each gene fragment cannot exceed the payload capacity of the drone.

The setting of the initial population has a significant impact on the search process of the genetic algorithm. A specific initial population can enhance the convergence speed of the algorithm [15]. To obtain a high-quality initial population, we formulate a package similarity measure and design a population initialization method based on agglomerative clustering. The similarity of package a and package b is calculated by the following equation:

$$Sim(a, b) = \frac{Lcs(path_a, path_b)}{Dist(dst_a, dst_b) + 1} \quad (14)$$

where $path_a$ and $path_b$ are the shortest paths from the source to the destination of two packages, respectively. $Lcs(path_a, path_b)$ is the proportion of the common part of $path_a$ and $path_b$. $Dist(dst_a, dst_b)$ denotes the length of the shortest path between the destinations of two packages. The similarity of cluster C_1 and cluster C_2 is the average similarity of the packages within the clusters:

$$Sim(C_1, C_2) = \frac{1}{|C_1| \cdot |C_2|} \sum_{P_i \in C_1, P_j \in C_2} Sim(P_i, P_j) \quad (15)$$

The pseudo-code of the population initialization method based on agglomerative clustering is shown in Algorithm 1. It consists of three phases: aggregation (Lines 2-8), division (Lines 9-11), and population generation (Lines 12-22). The initialization operation of the algorithm involves creating an empty tree-like clustering structure *tree*, a list of clustering solutions *cluster*, and an initial population *population* (Line 1). In the aggregation phase, each package in P is considered as an initial cluster, and a node is created in the *tree* for each cluster (Line 2). Then, the current clustering solution C and the similarity matrix M are initialized (Line 3). Finally, the two clusters with the highest similarity are merged into a cluster, followed by updates to C , M , and *tree*. The merging process continues until all packages are classified into one cluster (Lines 4-8). In the division phase, the *tree* obtained from the aggregation is divided at different levels to generate several clustering solutions (Lines 9-11). Function *divideTree* performs hierarchical traversal of the tree in a top-down manner. When a node is found for which the number of packages in the cluster is not greater than *num*. The cluster corresponding to that node is incorporated into the clustering solution, then the node and its descendants are deleted. The search process continues until a complete clustering solution is obtained (Line 10). The value range of parameter *num* is determined based on the number of packages. In the population generation phase, each solution in the *cluster* is used to generate chromosomes (Lines 12-22). The specific process is to use function *encode* to encode each cluster in *cluster*[i] into several gene fragments, which are then concatenated to form a complete chromosome (Lines 16-18). Function *encode* first randomly selects a type of drone, then iterates over the packages in cluster c and assigns the current package to that drone if it meets the payload limit. When a drone cannot add a new

Algorithm 1: Agglomerative Clustering Based Population Initialization Method.

Input: Package List P , Delivery network G , Drone type list D , Number of individuals N , Similarity function Sim
Output: Initialization population *population*

- 1: *tree*, *cluster*, *population* = {}
- Step 1. Aggregate packages into a tree by similarity**
- 2: Create nodes for each package in the *tree*
- 3: Initialize the current cluster C and similarity matrix M
- 4: **while** $len(C) > 1$ **do**
- 5: $maxClusters = argmax(M)$
- 6: $newCluster = maxClusters[0] \cup maxClusters[1]$
- 7: Update C , M and *tree*
- 8: **end while**
- Step 2. Divide tree to obtain clusters**
- 9: **for** $num = \frac{\sqrt{len(P)}}{2}$ to $2 * \sqrt{len(P)}$ **do**
- 10: $cluster.append(divideTree(tree, num))$
- 11: **end for**
- Step 3. Generate population based on clusters**
- 12: **for** $i = 1$ to $len(cluster) - 1$ **do**
- 13: $n = 0$
- 14: **while** $n < \frac{N}{len(cluster)}$ **do**
- 15: $chromosome = \{\}$
- 16: **for** any $c \in cluster[i]$ **do**
- 17: $chromosome = chromosome \parallel encode(c, D)$
- 18: **end for**
- 19: $population.append(chromosome)$
- 20: $n = n + 1$
- 21: **end while**
- 22: **end for**
- 23: **return** *population*

package, the drone is reselected until all packages have been assigned.

The time complexity of the method is discussed as follows. Assume that the number of packages is $|P|$, the time complexity of calculating the similarity matrix and each merge operation are both $O(|P|^2)$. Therefore, the time complexity of the aggregation phase is $O(|P|^3)$. The division phase performs a hierarchical traversal of the tree with a time complexity of $O(|P|^{\frac{3}{2}})$. In the population generation, assume that there are K clustering solutions in the cluster and the number of individuals is N . Each clustering solution generates $\frac{N}{K}$ individuals, thus the time complexity of this phase is $O(N \cdot |P|)$. In summary, the time complexity of the population initialization method is $O(|P|^3 + N \cdot |P|)$.

2) *Selection and Reproduction:* In selection, all individuals are sorted according to fitness values, and a fraction of individuals with high fitness values is retained. The remaining individuals are then determined to have a probability of being selected based on the order of sorting. The higher the fitness value of an individual, the higher the probability of being selected.

In the reproduction, the crossover operation and the neighborhood search each generates half of the offspring. The process of

crossover operation is as follows. First, two chromosomes A and B are randomly selected from the population as the parent generation. Then, several gene fragments are randomly selected from chromosome A and directly retained in the offspring. Finally, the existing packages in the offspring are deleted from chromosome B , and the remaining portions are added to the offspring. The neighborhood search algorithm improves the current solution by searching its neighboring solutions. We integrate this method into the reproduction process to improve the ADS-GA's local search capabilities. The process of generating offspring is as follows. First, it generates three neighborhood solutions by searching the neighborhoods of excellent individuals through three mutation operators. Then, compare the fitness values of the three solutions and add the solution with the highest fitness to the current population.

3) *Directed Mutation*: Mutation operators include exchange, insertion, and refactoring. After selecting the chromosomes to be mutated, the algorithm will randomly choose an operator. The exchange operator randomly exchanges two genes within a selected gene fragment. The insertion operator inserts a gene from a gene fragment into another gene fragment. If the insertion operation fails to meet the drone's payload limit, the gene is inserted into an empty gene fragment. The refactoring operator follows these steps. First, it selects several gene fragments from the chromosome. The genes within these fragments are added to a list. Then, it removes the selected gene fragments from the chromosome and randomize the order of the list. Finally, it uses the function *encode* to generate the new gene fragments and add them to the chromosome.

Upon the mutation of a chromosome, standard mutation operations cannot differentiate between different gene fragments. They will apply the operator to several randomly selected gene fragments. To expedite the elimination of poor-quality gene fragments, we define the fitness value of the gene fragment and design a directed mutation mechanism. The directed mutation is employed solely during the initial phases of algorithmic iteration. During a mutation operation, gene fragments within a chromosome are sorted in descending order according to their fitness values. The probability of being altered is then determined based on the order of sorting. Gene fragments with low fitness value are more likely to be altered. The fitness value of the gene fragment is determined using the equation below:

$$value = \frac{E_s}{E_m} \quad (16)$$

where E_s represents the energy required by delivering the packages in the gene fragment via the single-package delivery approach, and E_m represents the expected energy consumption of the gene fragment.

B. PIDC Method

PIDC comprehensively considers the impact of dynamic wind conditions, recharging, and waiting on the energy consumption of the drone. It consists of two phases: the static phase and the dynamic phase.

The static phase focuses on the planning before drone take-off, involving deterministic DaaS composition. We model the planning process in the static phase as a Markov Decision Process (MDP), using policy iteration to generate a policy. The policy is based on predicted wind conditions. It is used to make decisions such as path selection, recharging decision, and waiting decision for the drone during the delivery process. The MDP is defined as a 5-tuple $\langle S, A, DP, r, \gamma \rangle$. $S = \{s_1, s_2, \dots, s_n\}$ is a finite set of states. $s_i = \langle t, re, cn \rangle$ is the combination of current time, remaining battery power of the drone, and current node. $A = \{a_1, a_2, \dots, a_n\}$ is a finite set of actions, including flying to a certain adjacent station, recharge, and wait. The action space is variable, and the actions the drone can take depend on the current state of the drone. Assume that $A_i \in A$ is the set of optional actions of the drone in state s_i and N_i represents the set of adjacent nodes of node $s_i.cn$. In the drone delivery scenario, the state transition function $DP(s'|s, a)$ is deterministic. Thus, let $T(s, a)$ represent the new state arrived after state s performs action a . $r(s, a)$ is the reward function. The reward obtained by the drone flying to adjacent nodes is set to a negative value for energy consumption. The reward for recharging or waiting is set to a fixed value. γ is the discount factor, which takes a value in the range of $[0, 1]$. The policy is denoted by π , $\pi(a|s)$ represents the probability that the policy performs action a at state s . $V^\pi(s)$ denotes the state-value function based on the policy π in the MDP.

The dynamic phase encompasses the period from the drone take off to the completion of package delivery, i.e., the service implementation phase. Due to wind uncertainty, the policy learned based on predicted wind conditions in the static phase may be ineffective. For example, the changes in wind cause the energy consumption of the selected DaaS service to increase or exceed the remaining battery power of the drone. We address this challenge by conducting dynamic detection during the delivery process. When changes in wind conditions influence the policy's effectiveness, a near optimal adjacent node is selected based on $V^\pi(s)$.

The pseudo-code of PIDC is presented in Algorithm 2. The static and dynamic phases are executed at the cloud server and edge server, respectively. In the static phase, the cloud server obtains the policy $\pi(s)$ and state-value function $V(s)$ based on the predicted wind conditions by the policy iteration path planning algorithm (Algorithm 3) (Line 1). In the dynamic phase, the edge server first obtains current wind conditions and drone's states. Then, it composes the near optimal drone service based on the output of the static phase. The specific process is as follows. Whenever the drone arrives at a station, the edge server selects a near optimal action a_i (Line 4) from the optional actions by $\pi(s)$. If action a_i is to recharge or wait, the algorithm executes the action and updates the drone's state (Lines 5-6). If action a_i is to fly to an adjacent node, calculate the actual energy consumption of the drone according to current wind conditions (Line 8). Then, the edge server determines whether action a_i satisfies the flight conditions (Lines 9-10). If the flight conditions are satisfied, the selected service is added to the DaaS composition plan. Then, it executes action a_i and update drone's state (Lines 11-12). If changes in wind conditions cause

Algorithm 2: Policy Iteration Based DaaS Composition Method (PIDC).

Input: Delivery network G , Wind information W , Drone D , Source src , Destination dst , Total weight of packages ω , Start time t_s , Number of iterations $epoch$, Policy improvement weight θ , Detection threshold φ

Output: DaaS composition plan $Comp$

Step 1. Static stage

1: Execute policy iteration path planning algorithm (Algorithm 3) to obtain policy $\pi(s)$ and state-value function $V(s)$

Step 2. Dynamic stage

2: Edge server obtains the current state of the drone s_d
3: **while** $s_d.cn \neq dst$ **do**
4: $a_i = \operatorname{argmax}_{a \in A_d} \pi(a|s_d)$
5: **if** action a_i is to recharge or wait **then**
6: Execute action a_i and $s_d = T(s_d, a_i)$
7: **else**
8: Calculate actual energy consumption $E(a_i)$
9: $\Delta E = \frac{E(a_i)}{r(s_d, a)}$
10: **if** $\Delta E < \varphi$ and $E(a_i) \leq f(s_d.e)$ **then**
11: $Comp.append(DaaS(s_d, a_i))$
12: Execute action a_i and $s_d = T(s_d, a_i)$
13: **else**
14: $j = \operatorname{argmin}_{n \in N_d} (E(a_n) - V(T(s_d, a_n)))$
15: **if** $E(a_j) \leq f(s_d.e)$ **then**
16: $Comp.append(DaaS(s_d, a_j))$
17: Execute action a_j and $s_d = T(s_d, a_j)$
18: **else**
19: Execute recharging action a and $s_d = T(s_d, a)$
20: **end if**
21: **end if**
22: **end if**
23: **end while**
24: **return** $Comp$

the policy to fail, the edge server initiates dynamic processing (Lines 14-20). First, a near optimal adjacent node is selected based on the current wind conditions and the $V(s)$ (Line 14). Then, it is determined whether the energy required to fly to the near optimal node exceeds the remaining battery power of the drone. If the conditions are satisfied, the drone flies to the node (Lines 16-17). Otherwise, the drone recharges at the current node (Line 19). The process continues until the package reaches its destination. Finally, the algorithm returns a DaaS composition plan from the specified source to the destination (Line 24). The time complexity of PIDC depends on the policy iteration path planning algorithm of the static phase.

Algorithm 3 provides the specific details of the policy iteration path planning algorithm. First, policy $\pi(s)$ is initialized as a uniformly random policy, and the values of all states are set to 0 (Line 1). Then, a policy evaluation (Lines 3-12) and a policy improvement (Lines 13-21) are performed during each iteration. $\pi(s)$ and $V(s)$ are finally returned (Line 23). In policy evaluation, the algorithm first iterates through all the states. If

Algorithm 3: Policy Iteration Path Planning Algorithm.

Input: Delivery network G , Wind information W , Drone D , Source src , Destination dst , Total weight of packages ω , Start time t_s , Number of iterations $epoch$, Policy improvement weight θ

Output: policy $\pi(s)$, state-value function $V(s)$

1: Initialize policy $\pi(s)$ and state-value function $V(s)$
2: **for** $i = 0$ to $epoch - 1$ **do**
3: **for any** $s_i \in S$ **do**
4: **if** $s_i.cn == dst$ **then**
5: $V(s_i) = 0$
6: continue
7: **end if**
8: **for any** $a_j \in A_i$ **do**
9: $Q(s_i, a_j) = r(s_i, a_j) + V(T(s_i, a_j))$
10: $V(s_i) = V(s_i) + \pi(a_j|s_i) * Q(s_i, a_j)$
11: **end for**
12: **end for**
13: **for any** $s_i \in S$ **do**
14: $a_j = \operatorname{argmax}_{a \in A_i} Q(s_i, a)$
15: $\Delta p = 0$
16: **for any** $a_k \in A_i$ and $a_k \neq a_j$ **do**
17: $\Delta p = \Delta p + \pi(a_k|s_i) * (1 - \theta)$
18: $\pi(a_k|s_i) = \pi(a_k|s_i) * \theta$
19: **end for**
20: $\pi(a_j|s_i) = \pi(a_j|s_i) + \Delta p$
21: **end for**
22: **end for**
23: **return** $\pi(s), V(s)$

the node corresponding to the state is the destination, the value of the state is set to 0 (Lines 4-7). The values of the other states are computed by the Bellman equation (Lines 8-11). The value of an action is the sum of the reward obtained by the current state for performing the action and the value of the new state after performing the action (Line 9). After obtaining the state-value function, the policy improvement is used to improve the current policy. For each state s_i , an optional action a_j with the highest value is first selected (Line 14). Then the probability that the current policy selects action a_j in state s_i is boosted. The procedure is as follows. First, it reduces the corresponding probabilities of the other optional actions in state s_i . Then, it calculates the sum Δp of the reduced probabilities of these actions (Lines 16-19). Parameter θ is the weight of the probability reduction and ranges from $[0, 1]$. Finally, it updates the value of $\pi(a_j|s_i)$ to be the sum of the probability of executing action a_j in the current policy and Δp (Line 20).

The time complexity of the algorithm is discussed as follows. In policy evaluation, the algorithm traverses the optional actions in all states and calculates the state-value function. Assume that the number of iterations is $epoch$, the size of the state space is $|S|$, and the average number of optional actions is $|A|$. The time complexity of policy evaluation is $O(epoch \cdot |S| \cdot |A|)$. The policy improvement also performs epoch times and iterates through the optional actions for each state. Therefore, the time

TABLE II
DRONE PARAMETERS

Parameter	Quadcopter	Hexacopter	Octocopter
Total weight [kg]	2.57	9	17
Payload capacity [kg]	1	3.5	7
Battery capacity [KJ]	450	1750	3500
Recharging time [min]	30	45	60
Flight speed [m/s]	10	12	15

complexity of policy improvement is also $O(\text{epoch} \cdot |S| \cdot |A|)$. In overall terms, the time complexity of the policy iteration path planning algorithm is $O(\text{epoch} \cdot |S| \cdot |A|)$.

V. EVALUATION

In this section, we first describe the settings for the simulation experiment. Then, we conduct a comprehensive evaluation on the performance of MW-DSP based on EXPRESS 2.0 which is an open source delivery service management system [36]. The simulation experiment first compares the delivery energy consumption of different service provisioning strategies. Then, the experiment compares the delivery energy consumption and iteration process of the relevant algorithms in the service sharing phase. In the service composition phase, comparison metrics include delivery energy consumption, delivery time and the robustness of the algorithm. Appendix B and Appendix C discuss the impact of the main parameters of ADS-GA on the optimization performance and analyze the scalability of MW-DSP.

A. Experimental Settings

We define a drone delivery network containing 40 stations based on Antwork's ADNET [32]. Each delivery station is equipped with three types of drones. The main parameters of drones are shown in Table II which are consistent with prior studies [13], [23], [35]. Drones can recharge at delivery stations when low on power. They can also choose to wait at a delivery station when encountering strong winds. We collected minute-level wind data from OpenWeather for the areas covering the delivery network [34]. These data are updated every 5 minutes. The frequency is determined based on the average time the drones fly between stations. The collected data includes predicted wind conditions and actual wind conditions. The predicted wind conditions are produced an hour in advance. The attribute information used includes location coordinates, year, month, day, hour, minute, wind speed, and wind direction. Regarding the package data, we randomly assign the destinations and set the weights of the packages using a normal distribution. For the fairness of comparison, each experiment is conducted 5 times with different package data to get the average result.

To the best of our knowledge, there are no existing methods which consider both phases of delivery service sharing and delivery service composition simultaneously. Therefore, we combine commonly used methods for each phase to compare with our MW-DSP. In this paper, we select representative methods for the routing problem and the service composition problem, including HGS [37], SA [38], RVNS [39], Shortest [25], A* [22], and

TABLE III
ALGORITHM PARAMETERS

Parameter	Value
Population size	100
Iterations of ADS-GA	200
Mutation probability	0.25
Number of time slots	20
Length of a time slot l_t [min]	5
Iterations of PIDC	40
Policy improvement weight θ	0.9
Detection threshold φ	1.25

SP-TAG [40]. We first compare the delivery energy consumption of different methods to verify the effectiveness of the MW-DSP strategy. Then, experiments are conducted to verify the effectiveness of the ADS-GA algorithm and PIDC method, respectively. The main parameter settings of the proposed method are shown in Table III. All the experiments are conducted on an Intel Core i7-12700 processor (2.10 GHz and 8.0 GB RAM) under Windows 11. All the algorithms are implemented in Python.

B. Experimental Results and Discussion

1) *Comparison of Service Provisioning Strategies*: In the first simulation experiment, we use HGS, SA, and RVNS for the service sharing phase, while Shortest, A*, and SP-TAG for the service composition phase. In addition, the energy consumption advantage of multi-package delivery is also verified by delivering all packages via single-package delivery as a baseline (named Single). We compare MW-DSP with 12 strategies formed by combining these methods in the delivery energy consumption. The delivery energy consumption refers to the energy required for drones to deliver all the packages of the station.

Table IV illustrates the delivery energy consumption for various strategies. It is evident from the table that as the number of packages increases, so does the delivery energy consumption. Notably, our MW-DSP strategy consistently exhibits lowest delivery energy consumption compared with alternative methods. This results in average energy savings ranging from 10.1% to 30.8%. Moreover, as the number of packages increases, the advantage of MW-DSP becomes more pronounced. For instance, when the number of packages is 20, MW-DSP can reduce energy consumption by 5.6% to 20.3% compared with other methods. When the number of packages is 100, the delivery energy consumption of MW-DSP is reduced by 11.4% to 32.4% compared with other methods. Furthermore, employing the same service composition method, the energy consumption of multi-package delivery is consistently smaller than that of single-package delivery. These experimental findings underscore the potential of MW-DSP to offer service plans with reduced energy consumption in the drone last-mile delivery system.

2) *Comparison of Service Sharing Algorithms*: As mentioned in Section IV, MW-DSP consists of two phases, so we verify the effectiveness of ADS-GA and PIDC separately. In this experiment, we evaluate the effectiveness of our ADS-GA

TABLE IV
EXPERIMENTAL RESULTS OF DELIVERY ENERGY CONSUMPTION ($\times 10^3$ KJ)

Strategy		Number of Packages				
Service Sharing	Service Composition	20	40	60	80	100
MW-DSP		20.698	34.714	55.416	75.402	95.502
HGS	Shortest	23.757	43.483	68.740	92.885	120.045
SA	Shortest	24.202	43.288	68.605	93.194	119.879
RVNS	Shortest	22.744	42.364	67.127	91.537	117.385
Single	Shortest	25.996	48.384	78.732	107.157	138.305
HGS	A*	23.543	43.801	68.861	92.993	120.957
SA	A*	23.971	43.649	69.081	94.007	120.686
RVNS	A*	22.594	42.832	66.441	92.012	119.358
Single	A*	25.990	49.293	80.620	110.018	141.285
HGS	SP-TAG	22.416	39.706	62.752	86.334	109.781
SA	SP-TAG	22.803	39.475	63.366	85.848	109.559
RVNS	SP-TAG	21.936	37.993	62.016	83.704	107.827
Single	SP-TAG	24.628	45.523	75.179	101.801	129.639

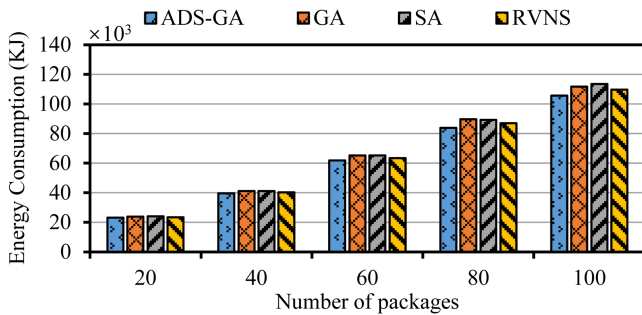


Fig. 3. Delivery energy consumption of different algorithms.

algorithm. The service sharing phase uses ADS-GA, HGS, SA, and RVNS respectively, while the service composition phase all uses the PIDC method. The experimental results are depicted in Fig. 3. Under the same experimental conditions, ADS-GA consistently exhibits the lowest delivery energy consumption. Compared with the other three methods, ADS-GA achieves average reductions in delivery energy consumption of 5.3%, 5.6%, and 2.9%, respectively. This is because the combination of crossover and neighborhood search effectively increases the search capability of our algorithm. Meanwhile, the population initialization method and directed mutation effectively expedite the iteration process of the algorithm. Fig. 4 shows the iteration process of several algorithms when the number of packages is 100 where the X-axis is the number of iterations and the Y-axis denotes the fitness value of the current optimal solution. It is evident that ADS-GA consistently outperforms other methods throughout the iteration process. Furthermore, the fitness value of the initial population optimal solution of ADS-GA is 39.3%, 46.2%, and 45.8% lower than the other three methods, respectively. This result also demonstrates the effectiveness of our population initialization method based on agglomerative clustering. Appendix B discusses the impact of the main parameters of ADS-GA on the optimization performance.

3) *Comparison of Service Composition Algorithms:* For this experiment, we evaluate the effectiveness of PIDC in terms of

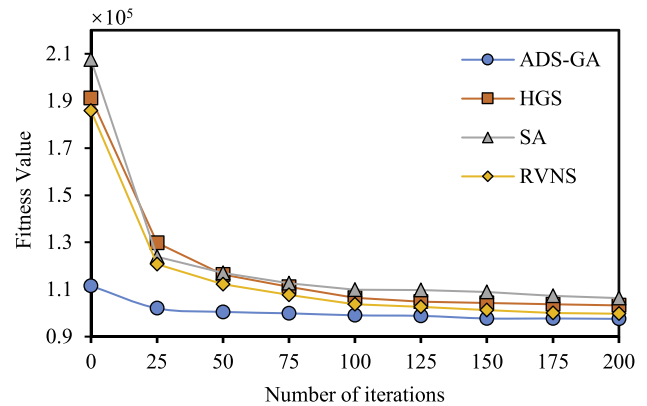


Fig. 4. Iteration process of different algorithms.

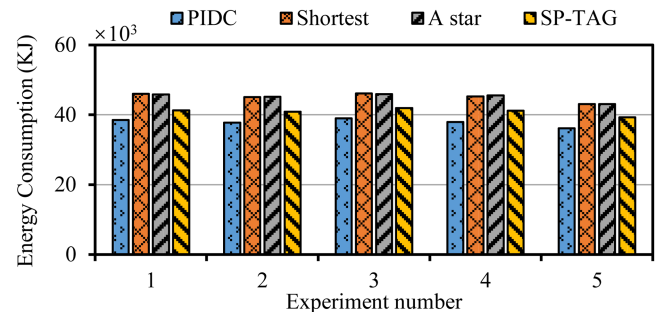


Fig. 5. Delivery energy consumption of different algorithms.

energy consumption, time, and robustness. In this experiment, ADS-GA is used for service sharing phases. PIDC, Shortest, A*, and SP-TAG are used for the service composition phase, respectively. We conducted 5 sets of experiments, with the number of packages set equal to the number of delivery stations. As illustrated in Fig. 5, the delivery energy consumption of PIDC is consistently lower than the other three methods by 16.0%, 16.1%, and 7.4%, respectively. This is primarily because other methods fail to adequately consider the impact of dynamic

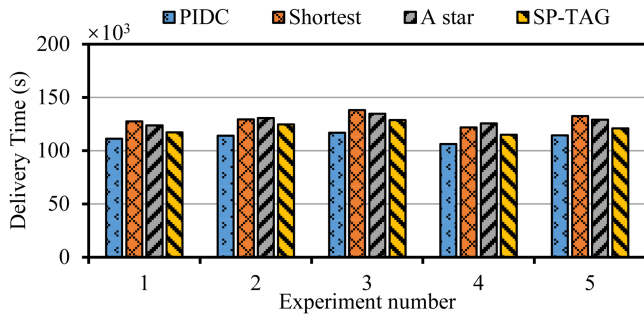


Fig. 6. Delivery time of different algorithms.

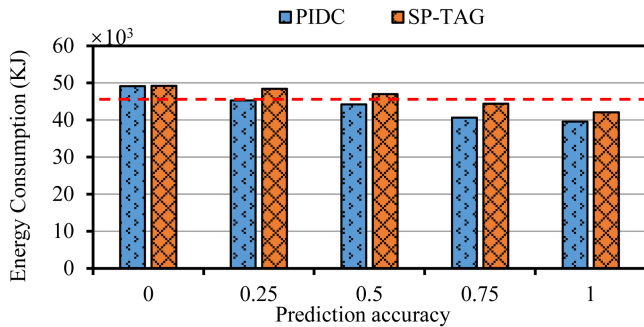


Fig. 7. Energy consumption with different prediction accuracy.

wind conditions, resulting in an unnecessary increase in energy consumption.

Fig. 6 shows the delivery time of the four methods. Our method exhibits the lowest delivery time, reduced by 13.3%, 12.6%, and 7.2% compared with other algorithms, respectively. It is noteworthy that Shortest has the minimum flight time by following the shortest path for delivery. However, higher energy consumption leads to longer recharging time, which in turn leads to longer delivery time. The experimental results indicate that our method has a superior performance in terms of energy consumption and time.

To evaluate the robustness of PIDC, we investigate the impact of the accuracy of the predicted wind conditions on the effectiveness of the method. In our experiments, we generate several sets of predicted wind conditions by randomly modifying some of the actual wind conditions. The prediction accuracy represents the proportion of the predicted wind conditions that match the actual wind conditions. Fig. 7 shows the delivery energy consumption of the PIDC and SP-TAG where the X-axis represents the prediction accuracy and the Y-axis is the delivery energy consumption. Each final result is the average of the experimental results from 5 sets of predicted wind conditions. The delivery energy consumption of Shortest and A* remains unaffected by the predicted wind conditions. Therefore, the smaller value of the two algorithms is represented by the red dashed line in the figure. As illustrated in the figure, the delivery energy consumption of PIDC and SP-TAG gradually increases when the accuracy decreases. Our method is consistently better than SP-TAG. When the accuracy drops below or equals 0.5, the delivery energy consumption of SP-TAG surpasses the optimal value of Shortest and A*. However, even at an accuracy of

0.25, our method still yields comparable results to the other two algorithms. This is because the dynamic processing of PIDC can reduce the impact of changes in wind conditions on the energy consumption of drones.

VI. CONCLUSION AND FUTURE WORK

In this paper, we investigated the multi-package drone delivery problem in an edge-based last-mile drone delivery system, with a special focus on the impact of dynamic wind conditions on the energy consumption of drones. In order to develop efficient service provisioning plans, we first established a spatio-temporal DaaS (Drone-as-a-Service) model based on service sharing. Then, we proposed a wind-aware DaaS service provisioning strategy for multi-package delivery (named MW-DSP) to minimize the delivery energy consumption. Meanwhile, we proposed an improved genetic algorithm (named ADS-GA) for service sharing and a policy iteration based DaaS composition method (named PIDC) to compose near optimal DaaS services. Comprehensive experimental results demonstrated that our strategy can effectively reduce the energy consumption of drones, and outperformed many other representative strategies. In our future work, we will explore the impact of robustness under uncertain wind conditions and availability of recharging equipment to relax the current assumptions. Meanwhile, we will investigate delivery service selection methods in resource-constrained environments and develop lightweight learning-based models for online service selection.

ACKNOWLEDGMENT

Special thanks to Antwork Robotics Company, Ltm.

REFERENCES

- [1] V. Garg, S. Niranjana, V. Prybutok, T. Pohlen, and D. Gligor, "Drones in last-mile delivery: A systematic review on efficiency, accessibility, and sustainability," *Transp. Res. Part D: Transport Environ.*, vol. 123, pp. 1–25, 2023.
- [2] M. Rahmani, F. Delavernhe, S. Senouci, and M. Berbineau, "Toward sustainable last-mile deliveries: A comparative study of energy consumption and delivery time for drone-only and drone-aided public transport approaches in urban areas," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 17520–17532, Nov. 2024.
- [3] A. Woodworth, "The wing delivery network," 2023, Accessed: Nov. 5, 2024. [Online]. Available: <https://blog.wing.com/2023/03/the-wing-delivery-network.html>
- [4] B. Shahzaad, A. Bouguettaya, S. Mistry, and A. Neiat, "Composing drone-as-a-service (DaaS) for delivery," in *Proc. IEEE Int. Conf. Web Serv.*, 2019, pp. 28–32.
- [5] Y. Choi and P. Schonfeld, "Optimization of multi-package drone deliveries considering battery capacity," in *Proc. 96th Annu. Meeting Transp. Res. Board*, 2017, pp. 8–12.
- [6] J. Xu, X. Liu, X. Li, L. Zhang, J. Jin, and Y. Yang, "Energy-aware computation management strategy for smart logistic system with MEC," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8544–8559, Jun. 2022.
- [7] J. Xu, X. Liu, X. Li, L. Zhang, and Y. Yang, "EXPRESS: An energy-efficient and secure framework for mobile edge computing and blockchain based smart systems," in *Proc. 35th IEEE/ACM Int. Conf. Autom. Softw. Eng.*, 2020, pp. 1283–1286.
- [8] A. Raivi, S. Huda, M. Alam, and S. Moh, "Drone routing for drone-based delivery systems: A review of trajectory planning, charging, and security," *Sensors*, vol. 23, no. 3, pp. 1–26, 2023.
- [9] B. Alkous, B. Shahzaad, and A. Bouguettaya, "Service-based drone delivery," in *Proc. IEEE 7th Int. Conf. Collaboration Internet Comput.*, 2021, pp. 68–76.

- [10] P. Tu, N. Dat, and P. Dung, "Traveling salesman problem with multiple drones," in *Proc. 9th Int. Symp. Inf. Commun. Technol.*, 2018, pp. 46–53.
- [11] S. Cavani, M. Iori, and R. Roberti, "Exact methods for the traveling salesman problem with multiple drones," *Transp. Res. Part C: Emerg. Technol.*, vol. 130, pp. 1–26, 2021.
- [12] D. Lei and X. Chen, "An improved variable neighborhood search for parallel drone scheduling traveling salesman problem," *Appl. Soft Comput.*, vol. 127, pp. 1–11, 2022.
- [13] K. Dorling, J. Heinrichs, G. Messier, and S. Magierowski, "Vehicle routing problems for drone delivery," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 47, no. 1, pp. 70–85, Jan. 2017.
- [14] W. Chiang, Y. Li, J. Shang, and T. Urban, "Impact of drone delivery on sustainability and cost: Realizing the UAV potential through vehicle routing optimization," *Appl. Energy*, vol. 242, pp. 1164–1175, 2019.
- [15] X. Yuan, J. Zhu, Y. Li, H. Huang, and M. Wu, "An enhanced genetic algorithm for unmanned aerial vehicle logistics scheduling," *IET Commun.*, vol. 15, no. 10, pp. 1402–1411, 2021.
- [16] M. Sajid, H. Mittal, S. Pare, and M. Prasad, "Routing and scheduling optimization for UAV assisted delivery system: A hybrid approach," *Appl. Soft Comput.*, vol. 126, pp. 1–21, 2022.
- [17] F. Sorbelli, F. Coro, S. K. Das, and C. M. Pinotti, "Energy-constrained delivery of goods with drones under varying wind conditions," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 9, pp. 6048–6060, Sep. 2021.
- [18] F. B. Sorbelli, F. Corò, L. Palazzetti, C. M. Pinotti, and G. Rigoni, "How the wind can be leveraged for saving energy in a truck-drone delivery system," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4038–4049, Apr. 2023.
- [19] J. Shi, H. Mao, Z. Zhou, and L. Zheng, "Adaptive large neighborhood search algorithm for the unmanned aerial vehicle routing problem with recharging," *Appl. Soft Comput.*, vol. 147, pp. 1–14, 2023.
- [20] M. Mahdi and D. Le, "Multiple traveling salesman problem with a drone station: Using multi-package payload compartments," in *Proc. Asian Conf. Intell. Inf. Database Syst.*, 2024, pp. 226–237.
- [21] B. Shahzaad, A. Bouguettaya, and A. Neiat, "Constraint-aware drone-as-a-service composition," in *Proc. Int. Conf. Service-Oriented Comput.*, 2019, pp. 369–382.
- [22] B. Alkous, A. Abusafia, A. Lakhdari, and A. Bouguettaya, "In-flight energy-driven composition of drone swarm services," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1919–1933, May/Jun. 2023.
- [23] B. Shahzaad, A. Bouguettaya, and S. Mistry, "Robust composition of drone delivery services under uncertainty," in *Proc. IEEE Int. Conf. Web Serv.*, 2021, pp. 675–680.
- [24] B. Shahzaad, A. Bouguettaya, S. Mistry, and A. Neiat, "Resilient composition of drone services for delivery," *Future Gener. Comput. Syst.*, vol. 115, pp. 335–350, 2021.
- [25] L. Chu, X. Li, J. Xu, A. G. Neiat, and X. Liu, "A holistic service provision strategy for drone-as-a-service in MEC-based UAV delivery," in *Proc. IEEE Int. Conf. Web Serv.*, 2021, pp. 669–674.
- [26] A. Hamdi, F. D. Salim, D. Y. Kim, A. G. Neiat, and A. Bouguettaya, "Drone-as-a-service composition under uncertainty," *IEEE Trans. Serv. Comput.*, vol. 15, no. 5, pp. 2685–2698, Sep./Oct. 2022.
- [27] B. Shahzaad and A. Bouguettaya, "Service-oriented architecture for drone-based multi-package delivery," in *Proc. IEEE Int. Conf. Web Serv.*, 2022, pp. 103–108.
- [28] J. Lima, R. Rocha, and F. Costa, "An approach for QoS-aware selection of shared services for multiple service choreographies," in *Proc. IEEE Symp. Service-Oriented Syst. Eng.*, 2016, pp. 221–230.
- [29] C. Zhang, J. Ning, J. Wu, and B. Zhang, "A multi-objective optimization method for service composition problem with sharing property," *Swarm Evol. Comput.*, vol. 49, pp. 266–276, 2019.
- [30] L. Ren, W. Wang, and H. Xu, "A reinforcement learning method for constraint-satisfied services composition," *IEEE Trans. Serv. Comput.*, vol. 13, no. 5, pp. 786–800, Sep./Oct. 2020.
- [31] D. Zhao, Z. Zhou, P. C. K. Hung, S. Deng, X. Xue, and W. Gaaloul, "CTL-based adaptive service composition in edge networks," *IEEE Trans. Serv. Comput.*, vol. 16, no. 2, pp. 1051–1065, Mar./Apr. 2023.
- [32] Company Antwork, 2019, Accessed: Nov. 5, 2024. [Online]. Available: <https://www.antwork.link/introduction.html>
- [33] A. Khosravi, R. Koury, L. Machado, and J. Pabon, "Prediction of wind speed and wind direction using artificial neural network, support vector regression and adaptive neuro-fuzzy inference system," *Sustain. Energy Technol. Assessments*, vol. 25, pp. 146–160, 2018.
- [34] OpenWeatherMap, 2012, Accessed: Nov. 5, 2024. [Online]. Available: <https://openweathermap.org/>
- [35] J. Stolaroff, C. Samaras, E. O'Neill, A. Lubers, A. Mitchell, and D. Ceperley, "Energy use and life cycle greenhouse gas emissions of drones for commercial package delivery," *Nature Commun.*, vol. 9, no. 1, pp. 1–13, 2018.
- [36] J. Xu, X. Liu, W. Pan, X. Li, A. Yao, and Y. Yang, "EXPRESS 2.0: An intelligent service management framework for AIoT systems in the edge," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2023, pp. 2022–2025.
- [37] T. Stadler, J. Schrader, and J. Dünneberger, "A hybrid genetic algorithm for solving the VRP with pickup and delivery in rural areas," in *Proc. Hawaii Int. Conf. Syst. Sci.*, 2023, pp. 1–9.
- [38] İ. İlhan, "A population based simulated annealing algorithm for capacitated vehicle routing problem," *Turkish J. Elect. Eng. Comput. Sci.*, vol. 28, no. 3, pp. 1217–1235, 2020.
- [39] J. Kong, H. Wang, and M. Xie, "Autonomous delivery vehicle routing problem with drones based on multiple delivery modes," *Comput. Operations Res.*, vol. 179, pp. 1–24, 2025.
- [40] B. George, S. Kim, and S. Shekhar, "Spatio-temporal network databases and routing algorithms: A summary of results," in *Proc. Int. Conf. Adv. Spatial Temporal Databases*, 2007, pp. 460–477.



Jia Xu (Member, IEEE) received the bachelor's, master's, and PhD degrees in computer science and technology from the School of Computer Science and Technology, Anhui University, Hefei, China, in 2014, 2017, and 2022, respectively. He was a software engineer focusing on industrial projects and solutions in iFLYTEK Company, Ltd from 2017 to 2018. From 2022 to 2024, he was a postdoctoral researcher with the School of Computer Science and Technology, Anhui University. He is currently a lecturer with the School of Internet, Anhui University. His current research interests include MEC, workflow systems, cloud computing, and resource management.



Hao Liu received the bachelor's degree in network engineering in 2022 from the School of Internet, Anhui University, Hefei, China, where he is currently working toward the master's degree with the School of Computer Science and Technology. His current research interests include MEC, resource management, and service composition.



Xiao Liu (Senior Member, IEEE) received the bachelor's and master's degrees in information management and information system from the School of Management, Hefei University of Technology, Hefei, China, in 2004 and 2007, respectively, and the PhD degree in computer science and software engineering from the Faculty of Information and Communication Technologies, Swinburne University of Technology, Melbourne, Australia, in 2011. He was an associate professor with Software Engineering Institute, East China Normal University, Shanghai, China during 2013–2015. He is currently an associate professor with the School of Information Technology, Deakin University, Australia. His current research interests include software engineering, distributed computing, and data mining, with special interests in workflow systems, cloud/fog computing, and social networks.



Azadeh Ghari Neiat received the PhD degree in computer science from RMIT University, Australia, in 2017. She is currently a lecturer of information technology with Deakin University, Australia. Her current research interests include Internet of Things, spatio-temporal data analysis, crowdsourcing/crowdsensing, Big Data mining, and machine learning with applications in the smart city, smart home, and recommender systems.



Yun Yang (Senior Member, IEEE) received the PhD degree in computer science from the University of Queensland, Brisbane, QLD, Australia, in 1992. He is currently a full professor with the Department of Computing Technologies, Swinburne University of Technology, Melbourne, VIC, Australia. His current research interests include software technologies, cloud and edge computing, workflow systems, and service computing.



Xuejun Li (Member, IEEE) received the PhD degree in computer application technology from the School of Computer Science and Technology, Anhui University, Hefei, China, in 2008. He is currently a full professor with the School of Computer Science and Technology, Anhui University. His major research interests include MEC, workflow systems, cloud computing, and intelligent software.